人工知能 期末レポート 課題 12

03420414 小松麟太郎

2025 年 2 月 1 日

1 序論

本課題では，メタヒューリスティックスを用いた多目的最適化シミュレーターを作成した．MOABC（Multi-Objective Artificial Bee Colony）と MOGWO（Multi-Objective Gray Wolf Optimization）の 2 つのアルゴリズムを実装し，2 つの関数に対してどちらかのアルゴリズムを選択して最適化を行うことができるようにした．本レポートでは，2 つのアルゴリズムの概要，シミュレーターの使用方法，及び性能評価について述べる．

2 アルゴリズム

本章ではシミュレーターに用いた MOABC と MOGWO の 2 つのアルゴリズムについてそれぞれ説明する．

2.1 外部アーカイブ

メタヒューリスティクスを用いた多目的最適化アルゴリズムでは，探索によって得られた非劣解を集団内に保持することが困難である．そこで，集団とは別に外部アーカイブ（External Archive，EA）を用意し，探索中に得られた非劣解を記録・管理する．外部アーカイブを活用することで，過去の優れた解を保持しつつ，多様性を維持しながら最適化を進めることが可能となる．本課題で用いた MOABC と MOGWO も外部アーカイブを採用している．

外部アーカイブは，世代が 1 つ進むごとに更新される．新たに得られた解が既存の解よりも優れている場合，既存の解を外部アーカイブから削除し，新しい解を追加する．この更新を繰り返すことで，外部アーカイブには次第に非劣解が蓄積されていく．しかし，外部アーカイブのサイズが無制限では，計算コストが増大し，最適化の効率が低下する．そのため，外部アーカイブのサイズを一定数に制限し，新たな解を追加する際には，既存の解の一部を削除する必要がある．

外部アーカイブから解を削除する際には，多様性を維持するための戦略が求められる．本課題では，混雑距離（Crowding Distance）を指標として解の選択を行う．混雑距離とは，外部アーカイブ内の各解について，最近傍の解との目的関数値の差を総和したものである．この値が大きいほど，その解の周囲には比較的多くの解が存在することを示す．そのため，混雑距離が大きい解から順に削除することで，非劣解の分布を均等に保ち，多様性を確保することができる．

2.2 MOABC

MOABC[1] は，ABC（Artificial Bee Colony）を多目的最適化に拡張したアルゴリズムである．元の ABC では蜂の群れを Employed Bee，Onlooker Bee，Scout Bee の 3 つのグループに分け，それぞれの蜂が探索を行うことで最適解を探索するが，MOABC では全ての蜂を Onlooker Bee として扱い，外部アーカイブを蜜源として探索する．

Onlooker Bee の個体数を N，i 番目の個体のいる蜜源（関数の入力ベクトル）を $\boldsymbol{x}_i$ とする．まず，$\boldsymbol{x}_i$ をランダムに初期化し，そのうち非劣解を外部アーカイブに保存する．蜜源の更新は次のように行われる．まず，外部アーカイブからランダムに 2 つの要素 $\boldsymbol{e}_k$ と $\boldsymbol{e}_l$ を蜜源として選択する．そして，$\boldsymbol{x}_i$ の中からランダムに m 次元を選び，m 次元は $\boldsymbol{e}_k$，その他の次元は $\boldsymbol{e}_l$ をもとに新たに $\boldsymbol{v}_i$ を生成する．j 番目の次元における $\boldsymbol{v}_i$ の値 v_{ij} は以下のように計算される．

$$v_{ij} = \begin{cases} x_{ij} + \mathrm{rand}(0,2)(e_{kj} - x_{ij}) \\ x_{ij} + \mathrm{rand}(0,2)(e_{lj} - x_{ij}) \end{cases} \tag{1}$$

ここで，$\mathrm{rand}(a,b)$ は区間 $[a,b]$ の一様乱数を生成する関数である．$\boldsymbol{v}_i$ が $\boldsymbol{x}_i$ よりも優越している場合，Onlooker Bee は $\boldsymbol{x}_i$ から $\boldsymbol{v}_i$ に移動し，$\boldsymbol{v}_i$ を外部アーカイブに保存する．元の $\boldsymbol{x}_i$ が $\boldsymbol{v}_i$ よりも優越している場合は何もしない．そして，互いに優越していない場合は，$\boldsymbol{v}_i$ を外部アーカイブに保存し，Onlooker Bee は 1/2 の確率で $\boldsymbol{v}_i$ に移動する．最後に 2.1 節で述べた方法に従って，外部アーカイブの更新を行う．MOABC のアルゴリズムの疑似コードは以下のようになる．

```
世代数 = 1
蜜源 x_i (i = 1,...,N) をランダムに初期化
各蜜源の関数値を計算
非劣解を外部アーカイブ(EA)に保存
while 世代数 < 最大世代数 do
    for each Onlooker Bee do
        EA からランダムにベクトルを選択
        式(1)からベクトル v_i を生成
        v_iの関数値を計算
        EAとx_iを更新
    end for
    非劣解のみをEAに保持
    if EA のサイズが上限を超えた
        混雑距離を用いて解を削除
    end if
    世代数 = 世代数 + 1
end while
```

2.3 MOGWO

MOGWO[2] は，GWO（Gray Wolf Optimization）を多目的最適化に拡張したアルゴリズムである．GWO は Gray Wolf の群れの中から，アルファ，ベータ，デルタの 3 つの優秀な個体を選び，それぞれの位置をもとにして他の個体が探索を行うことで最適解を探索する．MOGWO では，外部アーカイブから，混雑距離の逆数で重みづけした確率に従って，アルファ，ベータ，デルタを選択する．こうすることによって，周囲にほかの個体が比較的少ない個体を選択することができ，多様性を保つことができる．群れの個体数を N，i 番目の個体のいる位置（関数の入力ベクトル）を $\boldsymbol{x}_i$，アルファ，ベータ，デルタをそれぞれ $\boldsymbol{e}_\alpha$，$\boldsymbol{e}_\beta$，$\boldsymbol{e}_\delta$ として，$\boldsymbol{x}_i$ を次の手順で更新する．まず，次のようにベクトル $\boldsymbol{a}_i$，$\boldsymbol{c}_i$，$(i = 1, 2, 3)$ を生成する．

$$\boldsymbol{a}_i = 2\boldsymbol{r}_i \odot \boldsymbol{a}' - \boldsymbol{r}_i \tag{2}$$

$$\boldsymbol{c}_i = 2\boldsymbol{r}'_i \tag{3}$$

ここで，$\boldsymbol{r}_i$，$\boldsymbol{r}'_i$ は区間 $[0, 1]$ の一様乱数を要素に持つベクトル．$\boldsymbol{a}'$ は世代が進むにしたがって各要素が 2 から 0 まで線形に減少するベクトルである．$\odot$ はアダマール積であり，要素ごとの積を表す．そして，$\boldsymbol{a}_i$，$\boldsymbol{c}_i$ を用いて，$\boldsymbol{x}_i$ から $\boldsymbol{v}_i$ を計算する．

$$\begin{aligned} \boldsymbol{d}_\alpha &= |\boldsymbol{c}_1 \odot \boldsymbol{e}_\alpha - \boldsymbol{x}_i| \\ \boldsymbol{d}_\beta &= |\boldsymbol{c}_2 \odot \boldsymbol{e}_\beta - \boldsymbol{x}_i| \\ \boldsymbol{d}_\delta &= |\boldsymbol{c}_3 \odot \boldsymbol{e}_\delta - \boldsymbol{x}_i| \end{aligned} \tag{4}$$

$$\begin{aligned} \boldsymbol{x}_\alpha &= \boldsymbol{e}_\alpha - \boldsymbol{a}_1 \odot \boldsymbol{d}_\alpha \\ \boldsymbol{x}_\beta &= \boldsymbol{e}_\beta - \boldsymbol{a}_2 \odot \boldsymbol{d}_\beta \\ \boldsymbol{x}_\delta &= \boldsymbol{e}_\delta - \boldsymbol{a}_3 \odot \boldsymbol{d}_\delta \end{aligned} \tag{5}$$

$$\boldsymbol{v}_i = \frac{\boldsymbol{x}_\alpha + \boldsymbol{x}_\beta + \boldsymbol{x}_\delta}{3} \tag{6}$$

ここで，$\boldsymbol{d}$ もベクトルであり，式 (4) の絶対値は，各要素ごとに計算される．この $\boldsymbol{v}_i$ を Gray Wolf の次世代の位置として更新する．$\boldsymbol{v}_i$ が外部アーカイブの非劣解である場合，$\boldsymbol{v}_i$ を外部アーカイブに保存する．最後に 2.1 節で述べた方法に従って，外部アーカイブの更新を行う．MOGWO のアルゴリズムの疑似コードは以下のようになる．

```
世代数 = 1
Gray Wolfの位置 x_i (i = 1,...,N) を初期化
各x_iの関数値を計算
非劣解を外部アーカイブ(EA)に保存
while 世代数 < 最大世代数 do
    for each Gray Wolf do
        EA からアルファ，ベータ，デルタを選択
        式(2)-(6)からベクトル v_i を生成
        v_iの関数値を計算
        EAとx_iを更新
    end for
    非劣解のみをEAに保持
    if EA のサイズが上限を超えた
        混雑距離を用いて解を削除
    end if
    世代数 = 世代数 + 1
end while
```

3 使用方法

本章では作成したシミュレーターの使用方法について説明する．シミュレーターは python で実装されており，GUI は customtkinter を用いて作成した．実行には customtkinter と tkinter が必要である．インストールされていない場合は，以下のコマンドでインストールする．

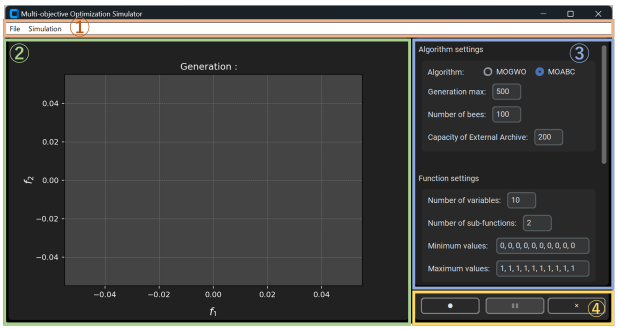

図 1: シミュレーターの画面

```
$ pip install tk
$ pip install customtkinter
```

図 1 は，シミュレーターのウィンドウである．ウィンドウは，1. メニューバー，2. グラフ画面，3. 設定画面，4. 実行・停止ボタンの 4 つの部分から構成されている．以下では，これらの画面の見方と操作方法について説明する．

3.1 メニューバー

ウィンドウの上部にあるメニューバーには，「File」，「Simulation」2 つのメニューがある．

- **File**
  - **Save Function Settings**
    入力されている関数とパレート最適解の設定を dat 形式で保存する．
  - **Load Function Settings**
    dat 形式で保存された関数とパレート最適解の設定を読み込む．
  - **Save Figure**
    グラフを保存する．グラフは png 形式または pdf 形式のどちらかを選択して保存することができる．
- **Simulation**
  - **Run Simulation**
    アルゴリズムを実行する．
  - **Pose Simulation**
    アルゴリズムの実行を一時停止する．

3.2 グラフ

ウィンドウ左側にあるグラフ画面には，アルゴリズムを実行して得られた外部アーカイブ内の非劣解の集合が表示される．横軸は目的関数 1 の値，縦軸は目的関数 2 の値であり，各非劣解は散布図としてプロットされる．また，パレート最適解の表示設定を有効にすればパレート最適解もグラフ上に点線で表示される．グラフ上部には，現在の反復回数が表示され，シミュレーションを一時停止すると被覆率（CR）も表示される．

3.3 設定

ウィンドウ右側にある設定画面でシミュレーションのパラメータや関数を設定することができる．主にアルゴリズムの設定 (Algorithm settings)，関数の設定 (Function Settings)．パレート最適解の設定 (True pareto front settings) の 3 つにわかれており，それぞれの設定を行うことでシミュレーションを実行できる．

3.3.1 アルゴリズムのパラメータ設定

- **Algorithm**
  使用するアルゴリズムを選択する．MOABC または MOGWO を選択できる．
- **Maximum Generation**
  最大世代数を整数値で設定する．世代数がこの値に達するとシミュレーションが終了する．
- **Population Size**
  Onlooker Bee や Gray Wolf の集団の個体数を整数値で設定する．
- **Capacity of External Archive**
  外部アーカイブの最大容量を整数値で設定する．外部アーカイブのサイズがこの値を超えると，混雑距離を用いて解を削除する．

3.3.2 関数の設定

ここでは，最適化する関数を設定する．このシミュレーターでは任意の数の変数を入力とした 2 つの目的関数 $f_1(\boldsymbol{x})$，$f_2(\boldsymbol{x})$ を設定することができる．

- **Number of Variables**
  関数の変数の数を整数値で設定する．
- **Number of sub-functions**
  目的関数が複雑な形の場合，任意の個数のサブ関数 $g_i(\boldsymbol{x})$ を設定し，それらを組み合わせて目的関数を記述することができるようになっている．ここでは，サブ関数の個数を整数値で設定する．
- **Maximum Values**
  各変数の最大値を「,」区切りで入力する．
- **Minimum Values**
  各変数の最小値を「,」区切りで入力する．
- **g_i**
  サブ関数 $g_i(\boldsymbol{x})$ を入力する．$g_i(\boldsymbol{x})$ は添え字が自分よりも小さいサブ関数 g_j $(j < i)$ を用いて記述する

ことができる．

- **f_1，f_2**
  目的関数 $f_1(\boldsymbol{x})$，$f_2(\boldsymbol{x})$ を入力する．$f_1(\boldsymbol{x})$，$f_2(\boldsymbol{x})$ はサブ関数 $g_i(\boldsymbol{x})$ を用いて記述することができる．

数式の入力方法

数式は，数式は基本的に Python の構文に従って入力する．変数 $\boldsymbol{x} = (x_1, x_2, \cdots, x_n)$ は「x[1]」，「x[2]」，$\cdots$，「x[n]」として入力する．また，数式中には，Numpy に用いられる定数や関数を使用することができる．表 1 に数式中で使用可能な記号を示す．表 1 の Numpy の関数に加えて表 2 に示す関数も使用することができる．以下ではシミュレーターの入力例をいくつか示す．

表 1: 数式中で使用できる記号

記号	説明	記号	説明
pi	円周率	log2	底が 2 の対数
e	ネイピア数	sqrt	平方根
+	加算	power	指数
-	減算	arcsin	逆正弦関数
*	乗算	arccos	逆余弦関数
/	除算	arctan	逆正接関数
**	べき乗	sinh	双曲線正弦関数
//	切り捨て除算	cosh	双曲線余弦関数
abs	絶対値	tanh	双曲線正接関数
sin	正弦関数	arcsinh	双曲線逆正弦関数
cos	余弦関数	arccosh	双曲線逆余弦関数
tan	正接関数	arctanh	双曲線逆正接関数
exp	指数関数	ceil	天井関数
log	自然対数	floor	床関数
log10	常用対数	round	四捨五入

表 2: その他の関数・記号

記号	説明
N	変数の数
if(条件, x, y)	条件が真のとき x，偽のとき y
sum(a, b, h(x_i))	$\sum_{i=a}^{b} h(x_i)$ を計算する．式中には, 変数 x_i を「x[i]」と記述できる
product(a, b, h(x_i))	$\prod_{i=a}^{b} h(x_i)$ を計算する．式中には, 変数 x_i を「x[i]」と記述できる

入力例 1

以下のような 2 目的最適化問題があるとする．

最適化問題 1

$$
\begin{aligned}
&\text{Minimize} \quad f_1, f_2 \\
&\text{Subject to} \\
&\qquad f_1 = x_1^2 \\
&\qquad f_2 = (x_1 - 2)^2 \\
&\qquad x_1 \in [-1000, 1000]
\end{aligned}
$$

このとき，シミュレーターには次のように入力することができる．

シミュレーターの入力

```
Number of Variables: 1
Number of sub-functions: 0
Minimum Values: -1e3
Maximum Values: 1e3
f_1: x[1]**2
f_2: (x[1]-2)**2
```

入力例 2

今度は例 1 より複雑な目的関数を考えてみる．以下のような最適化問題が次で定義されているとする．

最適化問題 2

$$
\begin{aligned}
&\text{Minimize} \quad f_1, f_2 \\
&\text{Subject to} \\
&\qquad f_1 = x_1 \\
&\qquad f_2 = g \times h \\
&\qquad g = 1 + 10\frac{\sum_{i=2}^{N} x_i}{N-1} \\
&\qquad h = 1 - \left(\frac{f_1}{g}\right)^{0.25} - \frac{f_1}{g}\sin(10\pi f_1) \\
&\qquad x_i \in [0, 1], \quad (i = 1, \cdots, 10)
\end{aligned}
$$

この問題は目的関数 f_1 と f_2 が別の関数 g，h を使って定義されている．このような場合には，シミュレーター上にもサブ関数を用いて目的関数を記述することができる．このとき，関数の親子関係に注意する必要がある．問題では h が $f1$ を用いて定義されているが，本シミュレーターはサブ関数が $f1$ を参照することができないため，少し書き換える必要がある．また，h は g を用いて定義されているため，サブ関数の入力時には h が g を参照できるように添え字に注意しなければならない．以上を考慮したうえで，シミュレーターには次のように入力することができる．

シミュレーターの入力

```
Number of Variables: 10
Number of sub-functions: 2
Minimum Values: 0,0,0,0,0,0,0,0,0,0
Maximum Values: 1,1,1,1,1,1,1,1,1,1
g_1: 1+10*sum(2,N,x[i])/(N-1)
g_2: 1-(x[1]/g[1])**0.25-(x[1]/g[1])*
    sin(10*pi*x[1])
f_1: x[1]
f_2: g[1]*g[2]
```

3.3.3 パレート最適解の設定

パレート最適解がわかっている場合，パレート最適解を設定してグラフ上に表示させることができる．目的関数の設定とは異なり，パレート最適解は 1 変数関数として入力する．パレート最適解の設定は以下のように行う．

- **Show true pareto front**
  パレート最適解を表示するかどうかを設定する．
- **Minimum value**
  変数の最小値を入力する．
- **Maximum value**
  変数の最大値を入力する．
- **f_1, f_2**
  パレート最適解を入力する．目的関数と同様に，表 1 の数学記号を使用することができるが，1 変数なので，sum や product を使うことはできない．変数は「x[1]」として入力する．また，f[2] は f[1] を用いて記述することができる．

以下に入力例を示す．

入力例

最適化問題 2 のパレート最適解は $x_1 \in [0,1]$，$x_i = 0$ $(i \neq 1)$ のときであり，次のように表せる．

最適化問題 2 のパレート最適解

$$f_1 : x_1$$
$$f_2 : 1 - x_1^{0.25} - x_1 \sin(10\pi x_1)$$
$$x_1 \in [0,1]$$

これは次のようにしてシミュレーターに入力することができる．

シミュレーターの入力

```
Show true pareto front: True
Minimum Values: 0
Maximum Values: 1
f_1: x[1]
f_2: 1-x[1]**0.25-x[1]*sin(10*pi*x[1])
```

3.4 実行・停止ボタン

ウィンドウの右下には，3 つのボタンがあり，それぞれ左から，シミュレーション開始ボタン，真ん中がシミュレーションの一時停止・再開ボタン，アプリケーションの終了ボタンになっている．3.3 節で説明した設定を行った後，左側のボタンを押すことでシミュレーションを開始することができる．シミュレーションが開始されると，グラフ画面に非劣解の集合が表示され，アルゴリズムが実行される．シミュレーションを一時停止する場合は，中央のボタンを押すことで一時停止することができる．一時停止中に再開する場合は，再度中央のボタンを押すことで再開することができる．シミュレーターを使い終わったら，右側のボタンを押すことでアプリケーションを終了することができる．

4 性能評価

5 考察